

BulbaInvest

Учебная брокерская платформа

Техническая документация и база знаний

Версия 1.0

20 мая 2026 г.

Имитация брокерской экосистемы для учебных целей

Содержание

I	Техническое задание	4
1	Назначение системы	4
2	Цели разработки	4
3	Пользователи системы	4
4	Функциональные требования	5
5	Нефункциональные требования	6
5.1	Производительность	6
5.2	Надёжность	7
5.3	Безопасность	7
5.4	Масштабируемость	7
5.5	Сопровождаемость	7
5.6	Логирование и мониторинг	7
5.7	Хранение данных	7
5.8	Развёртывание	7
5.9	API и обработка ошибок	7
6	Требования к данным	8
7	Требования к API	8
7.1	Gateway и клиентский доступ	8
7.2	Пользователь и авторизация	9
7.3	Кошельки и портфель	9
7.4	Сделки и заявки	9
7.5	Сервис рынка и графиков	10
8	Ограничения и допущения	10
9	Критерии готовности	11
II	База знаний	12
10	Архитектура и границы сервисов	12
10.1	Microservices	12
10.2	Bounded context	12
10.3	API Gateway	12
10.4	Service contracts	13
10.5	Event-driven architecture	13
10.6	Практические правила	13

11 Trading domain	14
11.1 Основные сущности	14
11.2 Company trades и user trades	14
11.3 Order book	14
11.4 Matching	15
11.5 Reservation	15
11.6 Trade consistency	15
11.7 Average buy price	15
11.8 Практические правила	15
12 Хранение данных и моделирование	16
12.1 PostgreSQL как source of truth	16
12.2 Flyway и миграции	16
12.3 Exposed и SQL abstraction	16
12.4 Wallet model	17
12.5 Portfolio model	17
12.6 Trade history	17
12.7 Деньги и Decimal	18
12.8 Indexes и constraints	18
13 Redis, events и messaging	19
13.1 Redis как key-value storage	19
13.2 Pub/Sub	19
13.3 Redis Streams	19
13.4 Events и commands	20
13.5 Message contract	20
13.6 Idempotency	20
13.7 Практические правила	21
14 Time series и OHLCV	22
14.1 Почему нужна отдельная модель	22
14.2 ClickHouse	22
14.3 Batch inserts	22
14.4 OHLCV	23
14.5 Granularity	23
14.6 Latest price	23
14.7 TTL	23
14.8 Практические правила	23
15 Backend service stack	24
15.1 Ktor	24
15.2 Dependency Injection	24
15.3 Kotlin Coroutines и Flow	24
15.4 StatusPages и ошибки	25
15.5 HTTP clients: timeouts и retries	25
15.6 Go service structure	25
15.7 cgo и C boundary	26

16 Market simulation	27
16.1 Что симулируется	27
16.2 Tick loop	27
16.3 Inventory	27
16.4 Simulation engine	28
16.5 Go и C boundary	28
16.6 Publishing quotes	28
16.7 HTTP debug API	28
16.8 Практические правила	28
17 Operations и testing	29
17.1 Docker Compose	29
17.2 Configuration	29
17.3 Health checks	30
17.4 Observability	30
17.5 Testing strategy	30
17.6 Reliability checks	31
17.7 Практические правила	31
 III Диаграммы архитектуры	 32
18 Gateway	32
19 Domain Service	33
20 Market Service	34
21 Graph Service	35

Часть I

Техническое задание

1 Назначение системы

Система предназначена для учебной имитации брокерской платформы, в которой пользователь может авторизоваться, просматривать рынок акций, получать котировки, анализировать графики, пополнять учебный кошелек, покупать и продавать активы, создавать заявки и просматривать портфель.

Продукт состоит из Android-приложения и backend-сервисов, которые совместно моделируют работу инвестиционного приложения и брокерской инфраструктуры без подключения к реальной бирже и без реальных денежных операций.

2 Цели разработки

Цель разработки --- создать готовый MVP учебного стартап-продукта для демонстрации полного цикла работы брокерской платформы:

- мобильное приложение для клиента-инвестора;
- backend API для авторизации, профиля, кошельков, портфеля и сделок;
- сервис генерации и выдачи котировок;
- сервис хранения истории котировок и построения свечных графиков;
- механизм покупки и продажи акций;
- механизм P2P-заявок между пользователями;
- запуск системы в контейнеризированном окружении;
- проверка работы системы под учебной нагрузкой до 10 000 клиентов.

3 Пользователи системы

Роль	Описание
Клиент / инвестор	Основной пользователь Android-приложения. Авторизуется по email-коду, просматривает рынок, пополняет баланс, покупает и продаёт акции, управляет портфелем и заявками.
Мобильное приложение	Android-клиент, который взаимодействует с backend через HTTP API и WebSocket.
Backend-сервисы	Внутренние сервисы системы, обеспечивающие доменную логику, котировки, историю графиков и проксирование клиентских запросов.

Роль	Описание
Сервис рынка	Внутренний сервис, который имитирует биржу, обновляет котировки и хранит доступное количество акций компании.

4 Функциональные требования

ID	Требование	Описание
FR-01	Android-приложение	Пользователь работает с системой через мобильное Android-приложение.
FR-02	Авторизация по email-коду	Пользователь запрашивает код на email, подтверждает его и получает JWT-токен.
FR-03	Хранение токена на клиенте	Android-приложение сохраняет JWT и использует его для авторизованных запросов.
FR-04	Просмотр профиля	Пользователь может получить данные текущего аккаунта.
FR-05	Обновление профиля	Пользователь может изменить имя профиля.
FR-06	Просмотр кошельков	Пользователь видит свои учебные кошельки и доступный баланс.
FR-07	Пополнение учебного баланса	Пользователь может пополнить учебный кошелек для выполнения операций покупки.
FR-08	Управление кошельками	Система предусматривает создание кошелька, выбор кошелька по умолчанию и удаление кошелька.
FR-09	Просмотр портфеля	Пользователь видит список купленных активов, количество акций и среднюю цену покупки.
FR-10	Просмотр позиции по тикеру	Пользователь может получить информацию по отдельной позиции портфеля.
FR-11	Просмотр рынка	Пользователь видит список доступных тикеров и актуальные цены.
FR-12	Получение текущих котировок	Система выдаёт актуальные котировки акций через REST API и WebSocket.
FR-13	Автоматическое обновление котировок	Сервис рынка периодически обновляет цены и публикует обновления для клиентов и сервисов.
FR-14	Просмотр информации об активе	Пользователь может открыть экран конкретной акции и увидеть текущую цену и детали инструмента.
FR-15	Просмотр графика цены	Пользователь может просматривать свечной график цены по выбранному тикеру.
FR-16	История котировок	Система сохраняет историю котировок и отдаёт агрегированные OHLCV-свечи.
FR-17	Покупка акций у компании	Пользователь может купить акции у имитируемой компании/рынка при наличии средств и доступного inventory.
FR-18	Продажа акций компании	Пользователь может продать акции обратно компании при наличии позиции в портфеле.

ID	Требование	Описание
FR-19	Создание заявки на продажу	Пользователь может выставить sell order на продажу своих акций.
FR-20	Просмотр стакана заявок	Пользователь может просматривать активные заявки на продажу по тикеру.
FR-21	Покупка из стакана	Пользователь может купить акции из order book по заданной цене.
FR-22	Покупка конкретной заявки	Пользователь может купить выбранную заявку другого пользователя.
FR-23	Отмена заявки	Пользователь может отменить свою активную заявку на продажу.
FR-24	История сделок	Пользователь может получить историю своих операций покупки и продажи.
FR-25	Хранение пользовательских данных	Система хранит пользователей, кошельки, портфели, заявки и сделки в БД.
FR-26	Хранение текущих котировок	Система хранит актуальные котировки в Redis для быстрого чтения.
FR-27	Хранение истории котировок	Система хранит исторические котировки в ClickHouse.
FR-28	WebSocket-обновления	Android-приложение получает live-обновления котировок через WebSocket.
FR-29	API Gateway	Клиент обращается к единой входной точке, которая маршрутизирует запросы к backend-сервисам.
FR-30	Обработка ошибок	API возвращает структурированные ошибки, а клиент отображает сообщения пользователю.
FR-31	Межсервисная интеграция	Сервисы обмениваются данными через HTTP, Redis Pub/Sub, Redis Streams и общую Docker-сеть.
FR-32	Защита от повторной обработки	Операции изменения inventory используют idempotency key.
FR-33	Логирование	Сервисы ведут логи запросов, ошибок и фоновых операций.
FR-34	Запуск через Docker Compose	Backend и инфраструктура запускаются через Docker Compose.
FR-35	Нагрузочное тестирование	Система проверена учебной имитацией нагрузки до 10 000 клиентов.

5 Нефункциональные требования

5.1 Производительность

Система должна поддерживать учебную нагрузку до 10 000 клиентов. Для снижения нагрузки на backend live-котировки передаются через WebSocket, а история котировок записывается в ClickHouse батчами. Текущие котировки должны читаться быстро за счёт хранения последних значений в Redis.

5.2 Надёжность

Система должна проверять баланс пользователя, доступное количество акций, статус заявки и корректность входных данных перед выполнением сделки.

Операции с кошельками, портфелем, ордерами и сделками должны сохраняться атомарно в пределах доменной БД. Операции изменения inventory должны быть идемпотентными.

5.3 Безопасность

Пользовательские endpoints должны требовать JWT-авторизацию. JWT выдаётся только после подтверждения email-кода. Секреты, пароли и адреса сервисов должны передаваться через переменные окружения.

5.4 Масштабируемость

Система должна быть разделена на компоненты по ответственности: мобильный клиент, Gateway, доменная логика, сервис рынка, сервис графиков и инфраструктурные хранилища. Котировки должны распространяться через Redis Pub/Sub, чтобы несколько потребителей могли получать обновления независимо.

5.5 Сопровождаемость

Код должен быть разделён на слои: UI, repository, network client, routes, services, database, external integrations. API-контракты должны быть описаны в документации и соответствовать реализации. Миграции и схемы хранения должны храниться в репозитории.

5.6 Логирование и мониторинг

Все backend-сервисы должны иметь health endpoints. Система должна логировать HTTP-запросы, ошибки интеграции, ошибки обработки сообщений и фоновые операции.

5.7 Хранение данных

Пользовательские данные, кошельки, портфели, ордера и сделки хранятся в PostgreSQL. Текущие котировки и служебные временные данные хранятся в Redis. История котировок хранится в ClickHouse.

5.8 Развёртывание

Backend-сервисы и инфраструктура должны запускаться через Docker Compose. Android-приложение должно собираться как отдельный Gradle-проект. Конфигурация окружения должна задаваться через .env и application config.

5.9 API и обработка ошибок

API должно использовать JSON. Ошибки валидации должны возвращать понятное сообщение. Ошибки отсутствующих данных должны возвращать 404. Ошибки конфликтов бизнес-логики должны возвращать 409. Gateway должен корректно обрабатывать недоступность downstream-сервисов.

6 Требования к данным

Данные	Сущность / хранилище	Назначение
Пользователь	users	Хранение профиля клиента: id, email, имя, даты создания и обновления.
Кошелёк	user_wallets	Хранение учебного баланса, валюты, зарезервированной суммы и признака кошелька по умолчанию.
Портфель	portfolio_positions	Хранение позиций пользователя по тикерам, количества, зарезервированного количества и средней цены покупки.
Заявка	sell_orders	Хранение P2P-заявок на продажу акций, цены, количества и статуса.
Сделка	trades	Хранение истории исполненных операций: покупатель, продавец, тикер, количество, цена, сумма и тип сделки.
Текущая котировка	Redis market:stocks	Быстрое получение актуальной цены и доступного количества акций.
Событие котировки	Redis channel market.quotes.updated	Рассылка обновлений котировок между сервисами и клиентскими подписками.
История котировок	ClickHouse quotes	Хранение time-series данных для построения свечей и графиков.
Inventory компании	market_company_inventory	Хранение доступного количества акций у компании/рынка.
Код авторизации	Redis с TTL	Временное хранение email-кодов входа.
Idempotency result	Redis	Хранение результата операции inventory для защиты от повторной обработки.

7 Требования к API

7.1 Gateway и клиентский доступ

Метод	Путь	Назначение	Входные / Выходные данные
GET	/health	Проверка состояния Gateway	--- / {status: ok}
Any	/api/**	Проксирование клиентских запросов к backend	Method, path, query, headers, body / Ответ backend

Метод	Путь	Назначение	Входные / Выходные данные
GET	/api/tickers	Получение списка тикеров	JWT / Список тикеров
GET	/api/quotes/{ticker}	Получение котировки	JWT, ticker / Котировка
GET	/api/quotes/{ticker}	Получение свечей	JWT, ticker, granularity / OHLCV-свечи
WS	/ws/quotes	Live-подписка на котировки	tickers / JSON-массив котировок

7.2 Пользователь и авторизация

Метод	Путь	Назначение	Входные / Выходные данные
POST	/api/auth/code/request	Запрос кода авторизации	email / Статус отправки
POST	/api/auth/code/confirm	Подтверждение кода	email, code / JWT token
GET	/api/users/me	Получение пользователя	JWT / User
PATCH	/api/users/me	Изменение имени	JWT, name / User

7.3 Кошельки и портфель

Метод	Путь	Назначение	Входные / Выходные
GET	/api/wallets/me	Список кошельков	JWT / Список Wallet
POST	/api/wallets	Создание кошелька	JWT, currency / Wallet
POST	/api/wallets/{id}/make-default	Выбор кошелька по умолчанию	JWT, walletId / Wallet
DELETE	/api/wallets/{id}	Удаление кошелька	JWT, walletId / Статус
POST	/api/wallets/me/deposit	Полноление баланса	JWT, amount / Wallet
GET	/api/portfolio	Портфель пользователя	JWT / Список позиций
GET	/api/portfolio/{ticker}	Позиция по тикеру	JWT, ticker / Позиция

7.4 Сделки и заявки

Метод	Путь	Назначение	Входные / Выходные
POST	/api/trades/company/buy	Покупка у компании	ticker, quantity, walletId / Trade
POST	/api/trades/company/sell	Продажа компании	ticker, quantity, walletId / Trade
GET	/api/trades	История сделок	JWT / Список Trade
GET	/api/order-book/{ticker}	Стакан заявок	JWT, ticker / OrderBook
POST	/api/orders/sell	Создание заявки на продажу	ticker, quantity, price / SellOrder
POST	/api/orders/buy	Покупка из стакана	ticker, quantity, maxPrice, walletId / Статус
POST	/api/orders/sell/{id}	Покупка конкретной заявки	JWT, orderId / Статус
POST	/api/orders/sell/{id}	Отмена заявки	JWT, orderId / Статус
GET	/api/orders/my	Мои заявки	JWT / Список SellOrder

7.5 Сервис рынка и графиков

Метод	Путь	Назначение	Входные / Выходные
GET	/stocks	Текущие котировки	--- / Список котировок
GET	/stocks/{ticker}	Котировка по тикеру	ticker / Котировка
POST	/api/v1/inventory/decrease	Уменьшение inventory	ticker, quantity, idempotencyKey / Результат
POST	/api/v1/inventory/increase	Увеличение inventory	ticker, quantity, idempotencyKey / Результат
GET	/api/tickers	Тикеры с историей	--- / Список тикеров
GET	/api/quotes/{ticker}	Последняя цена	ticker / Quote
GET	/api/quotes/{ticker}/OHLCV	OHLCV-свечи	ticker, granularity / Candles

8 Ограничения и допущения

- Система является учебной имитацией брокерской платформы.
- Все денежные операции выполняются в учебной валюте и не связаны с реальными платежами.
- Котировки генерируются сервисом имитации рынка.
- Список активов ограничен набором тикеров, поддерживаемых сервисом рынка.

- Нагрузка до 10 000 клиентов рассматривается как учебная проверка масштабируемости MVP.
- Бизнес-логика упрощена по сравнению с реальной брокерской системой.
- Реальные юридические, биржевые и платёжные интеграции не входят в состав MVP.

9 Критерии готовности

MVP считается готовым, если:

- Android-приложение собирается и запускается;
- пользователь может пройти авторизацию по email-коду;
- приложение сохраняет JWT и выполняет авторизованные запросы;
- backend запускается через Docker Compose;
- все основные сервисы имеют рабочий health endpoint;
- пользователь видит кошелёк и портфель;
- пользователь может пополнить учебный баланс;
- пользователь видит список тикеров и актуальные котировки;
- WebSocket передаёт обновления котировок;
- пользователь может открыть экран акции и посмотреть график;
- пользователь может купить и продать акции у компании;
- пользователь может создать, просмотреть, купить и отменить P2P-заявку;
- история сделок сохраняется и доступна через API;
- данные сохраняются в PostgreSQL, Redis и ClickHouse;
- нагрузочное тестирование или имитация клиентских запросов до 10 000 клиентов выполнены;
- документация содержит техническое задание, OpenAPI-контракт и инструкции запуска.

Часть II

База знаний

10 Архитектура и границы сервисов

Архитектура backend-системы описывает не только список сервисов, но и границы ответственности между ними. Хорошая архитектура отвечает на вопросы: кто владеет данными, кто принимает решения, как сервисы общаются, что происходит при ошибке и где проходит внешний API.

10.1 Microservices

Microservice --- это самостоятельный сервис с понятной ответственностью, собственным lifecycle, конфигурацией, контрактами и часто собственной базой данных. Его размер менее важен, чем ясность границ.

Плохой microservice --- это просто часть монолита, вынесенная в отдельный процесс, но продолжающая делить таблицы, бизнес-логику и внутренние модели с другими частями системы. Хороший microservice владеет своим состоянием и предоставляет наружу явный contract.

В trading-системах естественно выделяются разные зоны ответственности:

- пользовательская доменная модель: users, wallets, portfolios, orders, trades;
- рыночная модель: quotes, available inventory, volatility, ticks;
- аналитическая модель: historical quotes, candles, aggregates;
- edge layer: внешний API, WebSocket, auth boundary, routing.

10.2 Bounded context

Bounded context --- это граница, внутри которой термины имеют устойчивый смысл. Например, price в market context может быть текущей simulated quote, а price в trade history --- уже зафиксированной ценой исполненной сделки. Это разные факты, хотя название одинаковое.

Главное правило bounded context: у каждого факта должен быть один владелец. Если два сервиса считают себя владельцами одного значения, возникает shared ownership. Это почти всегда приводит к расхождениям.

10.3 API Gateway

API Gateway --- единая входная точка для клиентов. Он скрывает внутреннюю топологию сервисов, маршрутизирует запросы, может проверять auth, добавлять request id, логировать edge-запросы, применять rate limiting и держать WebSocket endpoint.

Gateway полезен, когда клиенту не нужно знать адрес каждого сервиса. Клиент обращается к одному public API, а gateway проксирует запросы во внутреннюю сеть.

Но gateway не должен становиться вторым domain service. Его задача --- edge-логика, а не правила сделок. Если gateway начинает решать, можно ли пользователю купить акцию, это признак утечки business logic.

10.4 Service contracts

Contract --- это обещание сервиса. Он описывает endpoints, payloads, status codes, message schemas, retry behavior и idempotency rules.

Контракты бывают:

- HTTP contracts: REST endpoints, OpenAPI, status codes;
- message contracts: JSON payloads в broker или stream;
- WebSocket contracts: формат сообщений и правила подписки;
- operational contracts: health checks, readiness, timeouts.

В distributed system нельзя полагаться на <<мы знаем, как оно внутри устроено>>. Внутреннее устройство сервиса может меняться, но контракт должен оставаться стабильным или версионироваться.

10.5 Event-driven architecture

Event-driven architecture строится вокруг событий. Producer публикует факт, consumer реагирует. Например, market service публикует quotes updated, gateway отправляет обновление клиентам, graph service сохраняет историю.

Важно различать events и commands:

- **Event** сообщает, что что-то уже произошло: цена обновилась, order matched, user created. Event может читать много consumers.
- **Command** просит выполнить действие: уменьшить inventory, создать order, списать деньги. Command требует результата, обработки ошибок и часто idempotency.

Для live data подходят Pub/Sub-механизмы. Для критичных операций лучше использовать durable streams, message queues или HTTP-команды с idempotency key.

10.6 Практические правила

- Сначала определяют owners данных, потом выбирают протоколы взаимодействия.
- Сервис не должен напрямую писать в базу другого сервиса.
- Gateway не должен содержать core business logic.
- Contracts нужно документировать отдельно от внутренней реализации.
- Events подходят для распространения фактов, commands --- для действий с результатом.
- Если операция затрагивает несколько сервисов, нужно заранее продумать idempotency, retries и compensation.

11 Trading domain

Trading domain описывает пользователей, деньги, портфели, заявки, сделки и рыночные операции. Даже в учебной системе эти понятия нужно моделировать аккуратно, потому что ошибки быстро приводят к отрицательным балансам, двойной продаже или расхождению истории.

11.1 Основные сущности

- **User** --- участник системы.
- **Wallet** --- денежный счет пользователя. Содержит currency, amount и reserved amount.
- **Portfolio position** --- количество бумаг по конкретному ticker.
- **Order** --- заявка на покупку или продажу.
- **Trade** --- исполненная сделка.
- **Quote** --- текущая рыночная цена или snapshot рынка.
- **Inventory** --- количество бумаг, доступных у компании или market maker.

11.2 Company trades и user trades

Сделки можно разделить на два типа:

- **Company trade** --- пользователь покупает у компании или продаёт компании. Здесь участвует внешний market context: нужно проверить или изменить inventory.
- **User trade** --- пользователь покупает у другого пользователя. В упрощённой системе это можно выполнить целиком внутри domain database.

Разница важна для consistency. User trade может быть атомарной SQL transaction. Company trade часто становится distributed workflow, потому что затрагивает несколько сервисов.

11.3 Order book

Order book --- список заявок. В полной биржевой системе есть buy side и sell side. В упрощённой модели может быть только sell side: пользователи выставляют sell orders, а покупатели выбирают конкретный order или покупают из стакана по максимальной цене.

Order status обычно включает:

- OPEN;
- PARTIALLY_FILLED;
- MATCHED или FILLED;
- CANCELLED;
- EXPIRED.

11.4 Matching

Matching --- процесс подбора встречных заявок. Простая стратегия --- price-time priority: сначала лучшая цена; при одинаковой цене --- более ранняя заявка.

Для покупки из sell book выбираются самые дешёвые open sell orders, пока не будет набрано нужное quantity или пока цена не превысит max price.

11.5 Reservation

Reservation защищает от двойного расходования.

Если пользователь выставляет sell order, система резервирует quantity в portfolio. Эти бумаги нельзя продать повторно до отмены или исполнения order.

Если пользователь создаёт buy order, можно резервировать деньги в wallet. Это предотвращает ситуацию, где пользователь создал несколько заявок на одну и ту же сумму.

11.6 Trade consistency

Сделка должна сохранять инварианты:

- balance не уходит ниже нуля;
- reserved amount не превышает amount;
- reserved quantity не превышает quantity;
- trade history соответствует изменениям wallet и portfolio;
- order status соответствует фактическому исполнению.

Для локальных операций помогает SQL transaction. Для distributed операций нужны idempotency, retries, compensation и reconciliation.

11.7 Average buy price

average_buy_price показывает среднюю цену покупки позиции. При докупке бумаг пересчитывается weighted average:

$$\text{newAverage} = \frac{\text{oldQty} \times \text{oldAvg} + \text{buyQty} \times \text{buyPrice}}{\text{oldQty} + \text{buyQty}}$$

При продаже средняя цена обычно не меняется, уменьшается только quantity.

11.8 Практические правила

- Деньги считать только decimal types.
- Все изменения wallet, portfolio, order и trade history выполнять атомарно, если они находятся в одной базе.
- Не позволять пользователю купить собственный order.
- Не исполнять closed order.
- При distributed trade использовать idempotency key.
- Отдельно хранить текущее состояние и историю событий.

12 Хранение данных и моделирование

Data storage выбирают под workload. В backend-системе с trading domain обычно нужны разные типы хранения: relational database для транзакций, Redis для быстрых значений и сообщений, column-oriented database для аналитики.

12.1 PostgreSQL как source of truth

PostgreSQL хорошо подходит для доменной модели, где важны транзакции, constraints и consistency. Пользователи, кошельки, портфели, ордера и сделки требуют атомарных изменений.

Пример: при покупке order нужно:

1. списать деньги покупателя;
2. начислить деньги продавцу;
3. изменить портфель продавца;
4. изменить портфель покупателя;
5. обновить status order;
6. записать trade history.

Все эти шаги должны быть выполнены в одной transaction. Если один шаг падает, остальные должны откатиться.

12.2 Flyway и миграции

Миграции делают схему базы частью кода. Flyway применяет SQL-файлы в заданном порядке и хранит историю применённых миграций.

Преимущества миграций:

- схема воспроизводится в новом окружении;
- изменения БД проходят code review;
- можно понять, какая версия схемы нужна приложению;
- проще запускать локальные и тестовые окружения.

12.3 Exposed и SQL abstraction

Exposed --- Kotlin SQL library. Она позволяет описывать таблицы и писать типизированные queries. Важно помнить, что ORM/DSL не отменяет понимания SQL.

Даже при использовании Exposed нужно понимать:

- где открывается transaction;
- какие queries выполняются;
- какие indexes нужны;
- как работает isolation;
- где может возникнуть race condition.

12.4 Wallet model

Wallet хранит деньги пользователя. Минимальная модель:

- id;
- user_id;
- currency;
- amount;
- reserved_amount;
- is_default;
- timestamps.

reserved_amount нужен для операций, где деньги блокируются заранее. Например, если buy order создаётся до исполнения, сумму можно зарезервировать, чтобы пользователь не потратил её дважды.

12.5 Portfolio model

Portfolio position хранит количество бумаг по ticker:

- user_id;
- ticker;
- quantity;
- reserved_quantity;
- average_buy_price.

reserved_quantity нужна для sell orders. Если пользователь выставил акции на продажу, он не должен продать эти же акции повторно до отмены или исполнения order.

12.6 Trade history

Trade history --- журнал исполненных сделок. Он не заменяет текущее состояние wallet или portfolio, но нужен для истории пользователя, аудита, аналитики и сверок.

Trade обычно содержит:

- buyer;
- seller;
- ticker;
- quantity;
- execution price;
- total amount;
- trade type;
- created time.

12.7 Деньги и Decimal

Деньги нельзя считать через floating point. Для финансовой логики нужны decimal types: `BigDecimal` в приложении и `NUMERIC` или `DECIMAL` в базе.

Floating point допустим для симуляции графиков или неофициальных вычислений, но не для source of truth по балансу.

Практическое правило: все значения, которые влияют на wallet, portfolio и settlement, должны проходить через decimal.

12.8 Indexes и constraints

Constraints защищают данные:

- unique email;
- unique (user_id, ticker) для portfolio position;
- positive quantity;
- valid status;
- foreign keys.

Indexes ускоряют частые queries:

- orders by ticker and status;
- trades by user;
- wallets by user;
- quotes by ticker and timestamp.

13 Redis, events и messaging

Redis может быть cache, key-value storage, Pub/Sub broker и lightweight stream platform. В backend-системах его часто используют для live updates, временных значений, locks, rate limits и межсервисных сообщений.

13.1 Redis как key-value storage

Redis хорошо подходит для быстрых данных, которые часто читаются и не требуют сложных relational queries.

Примеры:

- последняя цена ticker;
- auth code с TTL;
- idempotency key;
- session metadata;
- feature flags;
- counters.

TTL особенно полезен для временных значений: кодов подтверждения, short-lived tokens, temporary locks.

13.2 Pub/Sub

Redis Pub/Sub --- механизм публикации сообщений в канал. Producer делает publish, subscribers получают message.

Подходит для:

- live quotes;
- UI updates;
- notification fan-out;
- internal real-time signals.

Ограничение Pub/Sub: он не durable. Если subscriber был выключен, он пропустит сообщение. Поэтому Pub/Sub нельзя использовать как единственный транспорт для критичной финансовой операции.

13.3 Redis Streams

Redis Streams --- append-only log сообщений. Streams поддерживают consumer groups, acknowledgements и чтение истории.

Streams лучше подходят для workflows, где важно не потерять command или result:

- domain command;
- market command;

- payment event;
- reconciliation task;
- asynchronous processing.

Consumer group позволяет нескольким consumers разделять нагрузку. Pending messages можно дочитывать после сбоя.

13.4 Events и commands

Важно различать semantic type сообщения:

- **Event** --- факт, который уже произошёл: quote updated, order matched, user created.
- **Command** --- просьба выполнить действие: reserve quantity, decrement inventory, create payout.

Для event часто достаточно fire-and-forget. Для command почти всегда нужен result, timeout, retry и idempotency.

13.5 Message contract

Сообщение в Redis --- это тоже contract. Нужно документировать:

- channel или stream name;
- schema payload;
- обязательные поля;
- timestamps и units;
- idempotency key;
- error shape;
- retry rules.

JSON удобен для старта, но schema должна быть стабильной. При изменениях лучше добавлять новые поля, а не ломать старые.

13.6 Idempotency

Idempotency означает, что повтор одной и той же операции не меняет результат повторно.

Это критично для commands. Если сервис отправил command, получил timeout, но downstream успел применить изменение, повтор command не должен применить его второй раз.

Типовая схема:

1. request содержит idempotencyKey;
2. receiver проверяет, видел ли этот key;
3. если key новый, операция применяется и результат сохраняется;
4. если key старый, возвращается сохранённый результат.

13.7 Практические правила

- Pub/Sub --- для live updates, где допустима потеря отдельного сообщения.
- Streams --- для задач, где сообщение нужно обработать надёжнее.
- Критичные commands должны иметь idempotency key.
- Schema сообщений должна быть документирована.
- Consumers должны уметь переживать повторную доставку.

14 Time series и OHLCV

Time series --- данные, привязанные ко времени. Котировки акций являются классическим примером: каждый tick имеет ticker, price и timestamp.

14.1 Почему нужна отдельная модель

Текущая цена и история цен --- разные задачи. Текущую цену удобно хранить в Redis, а историю --- в базе, оптимизированной под аналитические запросы.

История котировок быстро растёт. Если писать каждый tick в обычную transactional database, запросы графиков могут начать мешать доменным операциям.

14.2 ClickHouse

ClickHouse --- column-oriented database для аналитики. Она хорошо подходит для queries по большим наборам данных: фильтрация по ticker, группировка по времени, расчёт агрегатов.

Типовая таблица quotes:

- ticker;
- price;
- available_quantity;
- volatility;
- ts.

Для ClickHouse важны partition и order key. Для котировок часто используют partition by month и order by (ticker, ts).

14.3 Batch inserts

ClickHouse не любит слишком частые одиночные inserts. Поэтому events обычно накапливают в buffer и пишут batch.

Flush может быть:

- time-based: каждые N секунд;
- size-based: когда buffer достиг N строк;
- shutdown flush: перед остановкой сервиса.

Если insert упал, данные можно вернуть в buffer и повторить позже. Но нужно следить, чтобы buffer не рос бесконечно.

14.4 OHLCV

OHLCV candle --- агрегат цены за временной bucket:

- **Open** --- первая цена;
- **High** --- максимальная цена;
- **Low** --- минимальная цена;
- **Close** --- последняя цена;
- **Volume** --- объём.

В учебных системах volume часто означает количество ticks в bucket. В реальной бирже volume обычно означает количество проторгованных единиц.

14.5 Granularity

Granularity определяет размер candle:

- 1 second;
- 1 minute;
- 5 minutes;
- 1 hour;
- 1 day.

Для разных экранов нужны разные granularity. График за час может использовать minute candles, а график за месяц --- daily candles.

14.6 Latest price

Latest price можно получать из time-series database через сортировку по timestamp, но для real-time UI это не всегда оптимально. Часто latest quote хранят отдельно в Redis, а ClickHouse используют для истории.

14.7 TTL

История тиков может быть дорогой. TTL позволяет автоматически удалять старые данные, например старше 12 месяцев.

Иногда используют downsampling: сырые тики хранят недолго, а агрегированные candles --- дольше.

14.8 Практические правила

- Не смешивать transactional data и analytics workload без необходимости.
- Писать в ClickHouse batch inserts.
- Чётко определить, что значит volume.
- Хранить timestamps в UTC.
- Для UI использовать подходящую granularity.

15 Backend service stack

Backend service stack --- это набор технологий и практик, из которых собирается сервис: HTTP framework, dependency injection, serialization, database access, external clients, concurrency model, logging и testing.

15.1 Ktor

Ktor --- легковесный Kotlin framework для HTTP-сервисов. Он хорошо подходит для microservices, потому что даёт простой контроль над routing, plugins, serialization, authentication и WebSockets.

Типовой Ktor service состоит из:

- application module;
- plugins: serialization, status pages, authentication, CORS, WebSockets;
- routes: HTTP endpoints;
- service layer: use cases и business logic;
- infrastructure layer: database clients, Redis clients, HTTP clients.

Routes не должны содержать много бизнес-логики. Их задача --- прочитать request, вызвать use case и вернуть response. Чем тоньше route layer, тем проще тестировать систему.

15.2 Dependency Injection

Dependency Injection помогает отделить создание объектов от их использования. Service layer не должен сам создавать HTTP client, Redis pool или database connection. Он должен получать зависимости через constructor.

DI даёт несколько преимуществ:

- проще тестировать use cases через fake dependencies;
- проще менять implementation без переписывания business logic;
- конфигурация собирается в одном месте;
- меньше hidden coupling.

Koin --- популярный DI framework для Kotlin. Его mental model прост: в модуле объявляются singleton или factory dependencies, а приложение получает уже собранный dependency graph.

15.3 Kotlin Coroutines и Flow

Coroutines позволяют писать asynchronous code в последовательном стиле. Это важно для HTTP, Redis, WebSocket и background tasks.

Flow подходит для потоков данных во времени: live quotes, events, updates, subscriptions. Например, WebSocket endpoint может получать `Flow<List<Quote>>`, фильтровать его по tickers и отправлять клиенту только нужные updates.

Важное свойство Flow --- cancellation. Если клиент закрывает WebSocket, coroutine должна остановить collection и освободить ресурсы.

15.4 StatusPages и ошибки

HTTP API должен возвращать предсказуемые ошибки. В Ktor StatusPages позволяет централизованно превращать domain exceptions в HTTP responses.

Типовое соответствие:

- validation error → 400 Bad Request;
- entity not found → 404 Not Found;
- business conflict → 409 Conflict;
- unexpected failure → 500 Internal Server Error.

15.5 HTTP clients: timeouts и retries

Любой HTTP client в service-to-service communication должен иметь timeouts:

- connect timeout;
- request timeout;
- socket timeout.

Без timeouts зависший downstream может удерживать ресурсы и постепенно положить upstream-сервис.

Retries полезны только для безопасных операций. Если повторить non-idempotent command, можно дважды применить изменение. Поэтому команды, меняющие состояние, должны иметь idempotencyKey.

15.6 Go service structure

Go хорошо подходит для сервисов с background loops, network IO и простым concurrency model. Типовая структура:

- cmd/<service> --- entrypoint;
- internal/<domain> --- core logic;
- internal/<transport> --- HTTP, Redis, messaging;
- internal/<config> --- configuration;
- interfaces вокруг внешних зависимостей.

В Go удобно отделять core logic от infrastructure через interfaces. Например, service может зависеть от Publisher, Driver, Repository, а не от конкретного Redis или C binding.

15.7 cgo и C boundary

cgo позволяет Go-коду вызывать C-библиотеки. Это полезно, когда simulation engine или низкоуровневая библиотека написаны на C.

Главный риск cgo --- memory ownership. Если C выделяет память, должен быть понятный способ освободить её. Go wrapper обязан конвертировать C structs в Go models и корректно вызвать free-функцию.

Хорошая граница с C:

- C layer не знает про HTTP, Redis и пользователей;
- Go layer отвечает за orchestration и integration;
- memory ownership явно задокументирован;
- C tests и Go tests запускаются отдельно.

16 Market simulation

Market simulation --- искусственная модель рынка, которая генерирует котировки и доступное количество активов без подключения к реальной бирже. Она нужна для учебных систем, демо-приложений, нагрузочных сценариев и тестирования trading flow.

16.1 Что симулируется

Минимальная модель акции:

- ticker;
- current price;
- available quantity;
- volatility;
- updated time.

На каждом tick цена немного меняется. Изменение может быть случайным, но должно учитывать volatility и нижнюю границу цены.

16.2 Tick loop

Tick loop --- background process, который через заданный interval обновляет состояние рынка.

Типовой цикл:

1. дождаться timer tick;
2. обновить цены в simulation engine;
3. получить snapshot;
4. сохранить current quotes;
5. опубликовать quote update event.

Tick loop должен корректно останавливаться при shutdown, чтобы не оставлять незавершённые операции.

16.3 Inventory

Inventory показывает, сколько бумаг доступно у компании или market maker.

При покупке у компании inventory уменьшается. При продаже компании inventory увеличивается.

Inventory не должен смешиваться с portfolio пользователя. Это другой bounded context.

16.4 Simulation engine

Simulation engine можно вынести в отдельный слой или библиотеку. Он должен заниматься только математикой и состоянием рынка:

- update price;
- apply buy;
- apply sell;
- return snapshot.

Он не должен знать про HTTP, Redis, users, wallets или orders. Такое разделение делает engine переиспользуемым и легче тестируемым.

16.5 Go и C boundary

Go удобен для service orchestration: HTTP API, Redis IO, background loops, graceful shutdown. C может быть полезен для низкоуровневого simulation engine.

При использовании cgo важно явно определить memory ownership. Если C возвращает snapshot в heap memory, Go wrapper должен освободить его через C free-функцию после conversion.

16.6 Publishing quotes

Симулятор может публиковать quotes двумя способами:

- сохранить последнюю quote в Redis key/hash для быстрых reads;
- отправить event в Pub/Sub или stream для live consumers.

Так разные consumers получают данные под свой сценарий: gateway стримит UI, graph service пишет историю, domain service читает последнюю цену.

16.7 HTTP debug API

Даже если основной обмен идёт через Redis, полезно иметь небольшой HTTP API:

- health check;
- list stocks;
- stock by ticker;
- current snapshot.

Такой API помогает отлаживать симулятор вручную и писать integration tests.

16.8 Практические правила

- Simulation не должна быть source of truth для денег пользователя.
- Inventory должен иметь одного владельца.
- Quote update можно распространять через events.
- Commands на изменение inventory должны быть idempotent.
- C boundary должен иметь ясные правила освобождения памяти.

17 Operations и testing

Operations и testing отвечают за то, чтобы система не только работала на локальной машине, но и была понятной, проверяемой и диагностируемой.

17.1 Docker Compose

Docker Compose удобен для локального окружения. Он позволяет поднять сервисы и инфраструктуру одной командой:

- application services;
- PostgreSQL;
- Redis;
- ClickHouse;
- MailHog или другой SMTP simulator.

Compose описывает networks, ports, volumes, environment variables и dependencies.

Важно понимать ограничение `depends_on`: он задаёт порядок запуска containers, но не всегда гарантирует readiness приложения. Для баз данных и аналитических систем полезны healthchecks.

17.2 Configuration

Сервис не должен зашивать адреса баз, пароли, ports и intervals прямо в код. Эти значения должны приходить из config files или environment variables.

Типовые параметры:

- server port;
- database URL;
- Redis host/port;
- JWT secret;
- SMTP host;
- downstream service URLs;
- batch size;
- flush interval;
- retry settings.

Один и тот же image должен запускаться в разных окружениях за счёт конфигурации, а не пересборки.

17.3 Health checks

Health endpoint отвечает на вопрос: жив ли процесс. Readiness отвечает на вопрос: готов ли сервис принимать traffic.

Простая проверка /health может вернуть:

```
{ "status": "ok" }
```

Для production полезно разделять:

- liveness: процесс не завис;
- readiness: зависимости доступны;
- startup: сервис завершил initialization.

17.4 Observability

Observability строится на logs, metrics и traces.

Logs должны помогать понять:

- какой request пришёл;
- какой user или trade id участвовал;
- какой downstream был вызван;
- какой status code вернулся;
- сколько заняла операция;
- где произошла ошибка.

Metrics показывают состояние системы в числах: request rate, latency, error rate, queue lag, batch size, memory usage. Tracing помогает увидеть цепочку запроса через несколько сервисов.

17.5 Testing strategy

Testing strategy зависит от риска:

- **Unit tests** подходят для pure logic: matching algorithm, ticker normalization, money calculations, validation, JSON parsing.
- **Integration tests** нужны для работы с реальными зависимостями: PostgreSQL transactions, Redis messages, ClickHouse inserts and queries, HTTP downstream calls.
- **Contract tests** фиксируют API между сервисами: request/response schema, message payload, status codes, idempotency behavior.
- **End-to-end tests** проверяют пользовательский сценарий целиком, но они дороже и медленнее. Их должно быть меньше, чем unit и integration tests.

17.6 Reliability checks

Для distributed workflows важно тестировать не только happy path:

- downstream timeout;
- duplicate command;
- message redelivery;
- partial failure;
- invalid payload;
- closed order;
- insufficient funds;
- insufficient quantity.

17.7 Практические правила

- Все сервисы должны иметь health endpoint.
- Конфигурация должна приходить извне.
- Logs должны содержать correlation id или request id.
- Критичная бизнес-логика должна иметь unit tests.
- Интеграционные границы должны покрываться integration или contract tests.
- Distributed operations нужно проверять на retries и idempotency.

Часть III

Диаграммы архитектуры

18 Gateway

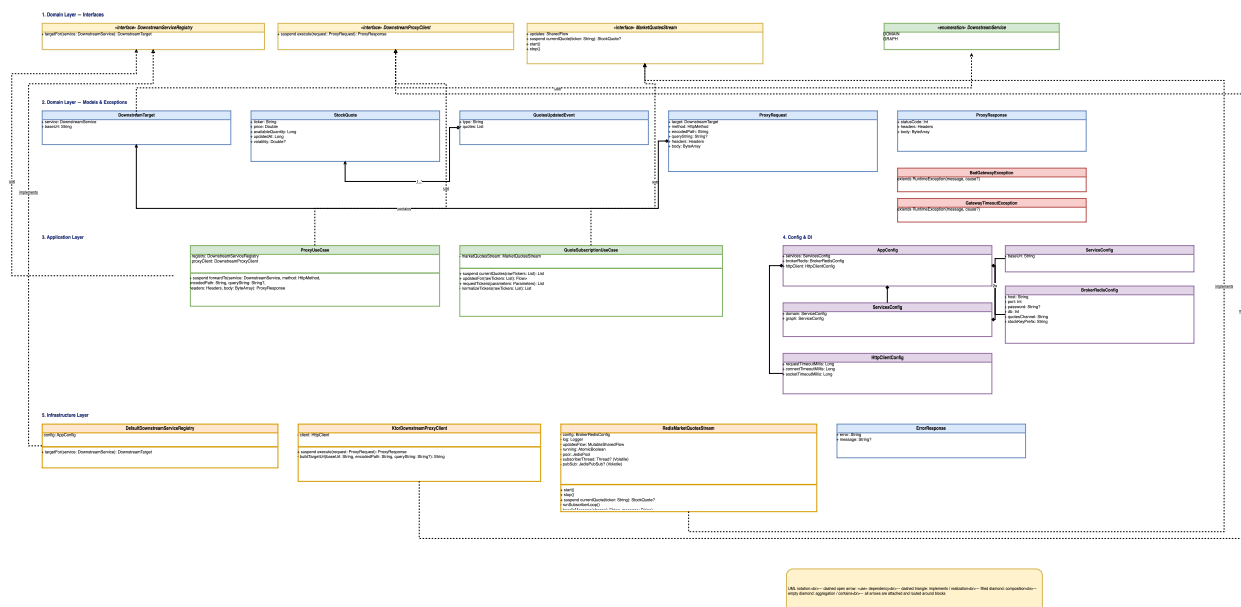


Рис. 1: Архитектура Gateway-сервиса

[illegible]

Рис. 2: Архитектура Domain Service

20 Market Service

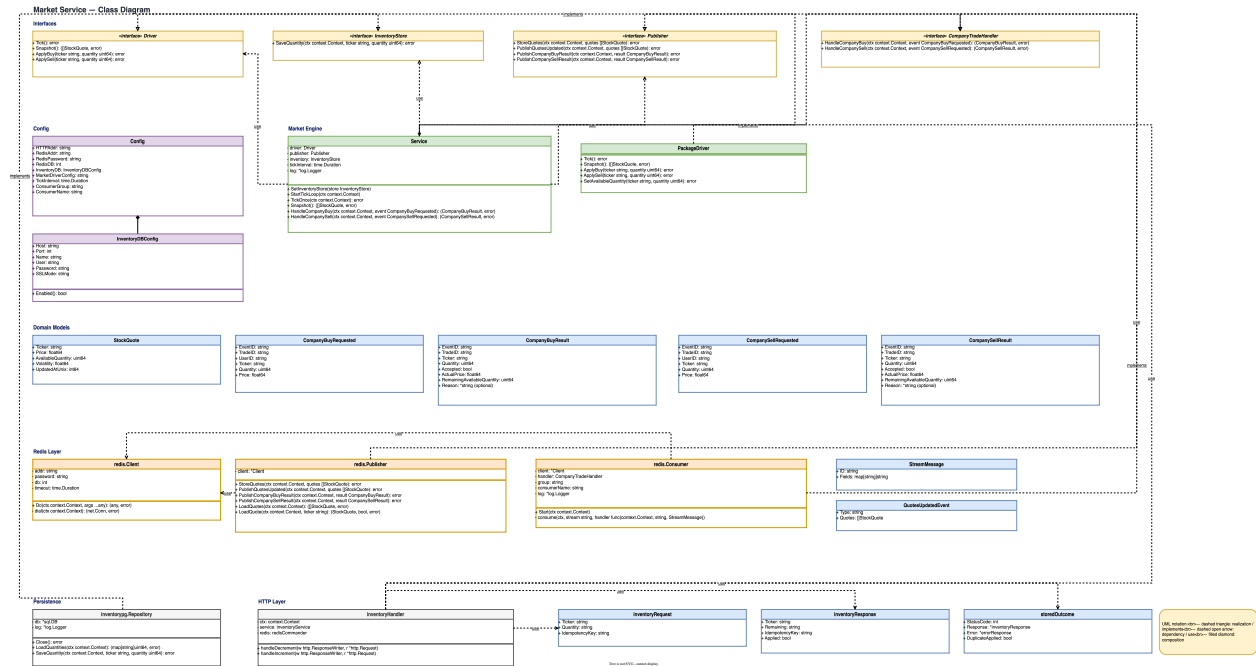


Рис. 3: Архитектура Market Service (симуляция рынка)

21 Graph Service

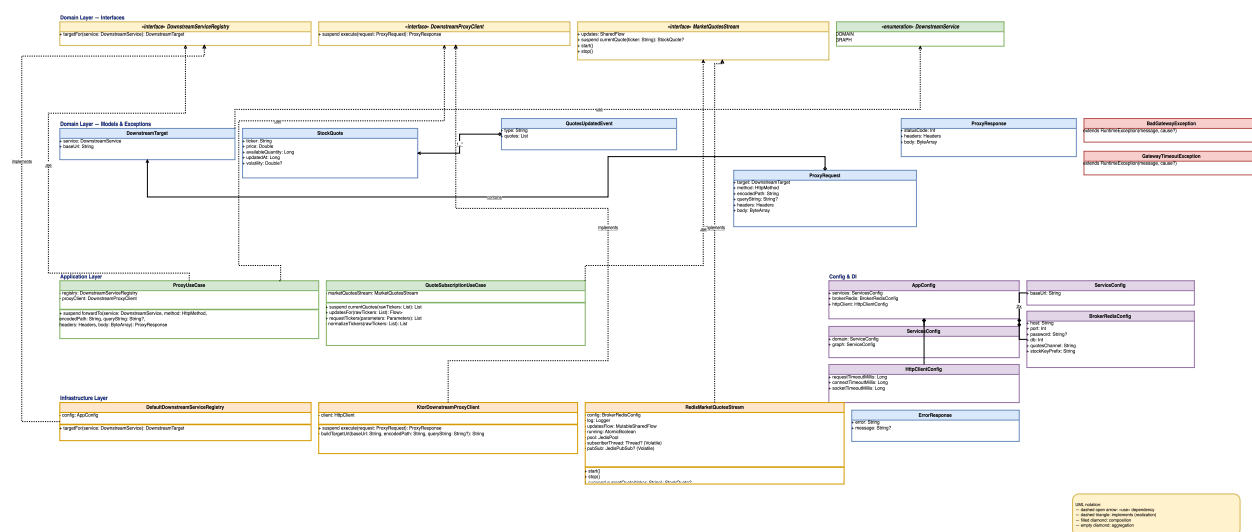


Рис. 4: Архитектура Graph Service (история котировок и графики)